

An abstract digital cityscape with glowing lines and data points, representing a complex network or data flow.

Ecommify: Diseño y Optimización de Base de Datos Híbrida

Arquitectura políglota para comercio electrónico de alto rendimiento

Presentado por

Valentina Rodríguez Romero

Andrés Santiago Santafe

Daniel Orlando Saavedra

Proyecto

Diseño Conceptual y Lógico de Base de Datos

Facultad de Ingeniería, Universidad de La Sabana

Contexto y Requisitos del Sistema

Ecommify es un **marketplace de alta escala** construido sobre el dataset público de Olist, diseñado para soportar millones de transacciones concurrentes con integridad garantizada.

Requisitos Transaccionales

- Consistencia estricta en **10M+ pedidos**
- Integridad absoluta en pagos e inventario
- Cero tolerancia a stocks negativos

Requisitos de Catálogo y Analítica

- Atributos variables por categoría de producto
- Análisis masivo de **15M+ reseñas**
- Disponibilidad continua del catálogo



Arquitectura Híbrida: Enfoque Políglota

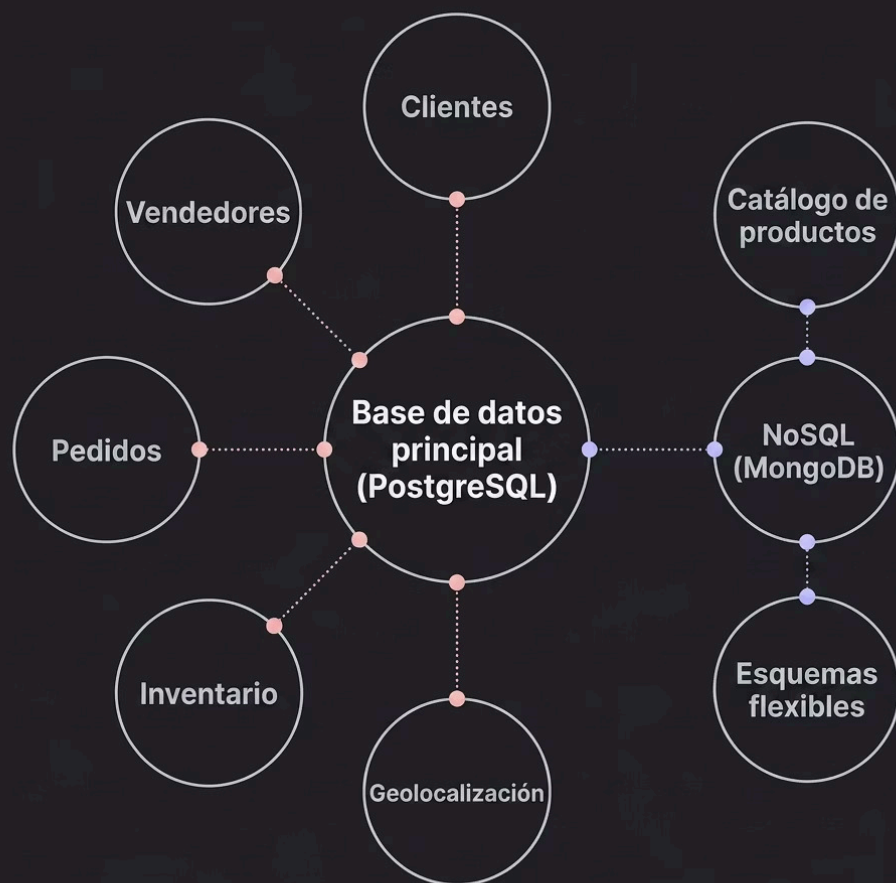
La combinación de **PostgreSQL + MongoDB** permite asignar cada dominio al motor más adecuado, maximizando consistencia donde es crítica y flexibilidad donde se requiere agilidad.

PostgreSQL — Core ACID

- Clientes, vendedores y pedidos
- Inventario y geolocalización
- Integridad referencial garantizada

MongoDB — Catálogo y Feedback

- Esquemas flexibles por categoría
- Análisis de reseñas aislado
- Alta disponibilidad de catálogo

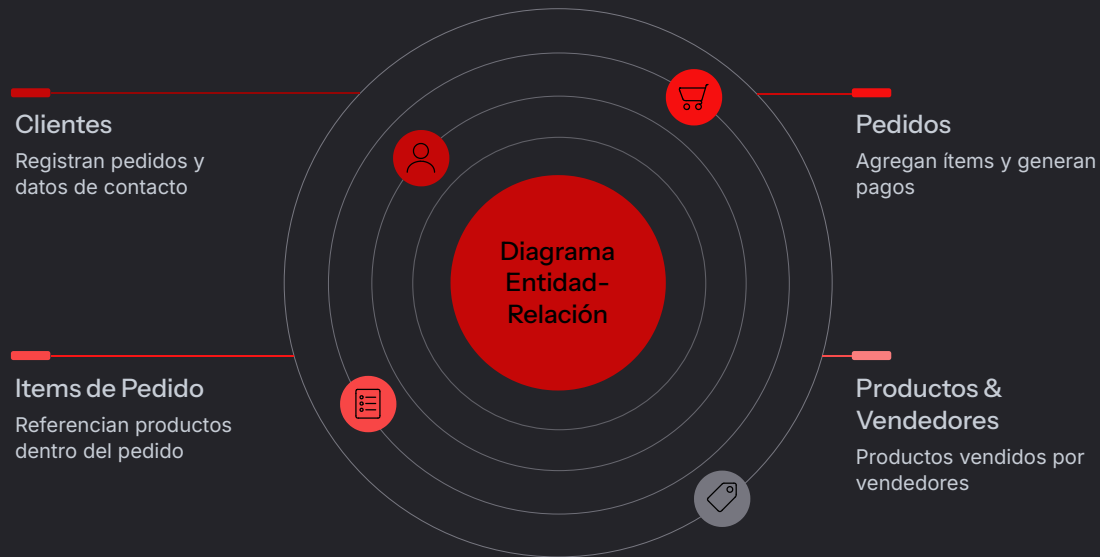


Principio de Diseño

Cada motor gestiona el dominio donde su modelo de datos ofrece la mayor ventaja técnica y operativa. No existe un único motor correcto: la **especialización es la estrategia**.

- ① La separación de dominios reduce la contención de recursos y permite escalar cada capa de forma independiente.

Modelo Conceptual: Diagrama Entidad-Relación



Normalización hasta 3FN

El modelo elimina redundancias y anomalías de actualización, garantizando que cada hecho se almacene en un único lugar.

Restricciones Clave

- Todo pedido debe tener **al menos un ítem** asociado
- Control estricto para **evitar stocks negativos** mediante triggers y constraints
- Pagos vinculados obligatoriamente a un pedido válido

Diseño Relacional Avanzado en PostgreSQL

PostgreSQL no se utiliza como un motor relacional básico: se aprovechan sus extensiones avanzadas para cubrir necesidades híbridas sin sacrificar consistencia ACID.



Tipos Avanzados

JSONB para especificaciones ligeras de productos y
Arrays nativos para gestión de fotos sin tablas auxiliares.



Extensión pg_trgm

Búsquedas de texto tolerantes a errores tipográficos mediante índices de trigramas. Mejora la experiencia de búsqueda del usuario final.



Extensión PostGIS

Cálculo nativo de distancias geográficas para estimación de costos de envío en tiempo real.



Particionamiento

Tabla `orders` particionada por rangos de fechas. Optimiza consultas históricas OLAP sin impactar el rendimiento OLTP.

Diseño Documental Flexible en MongoDB

MongoDB gestiona los dominios donde el esquema fijo sería una limitación operativa, aplicando patrones de diseño documentales para optimizar el rendimiento de lectura.

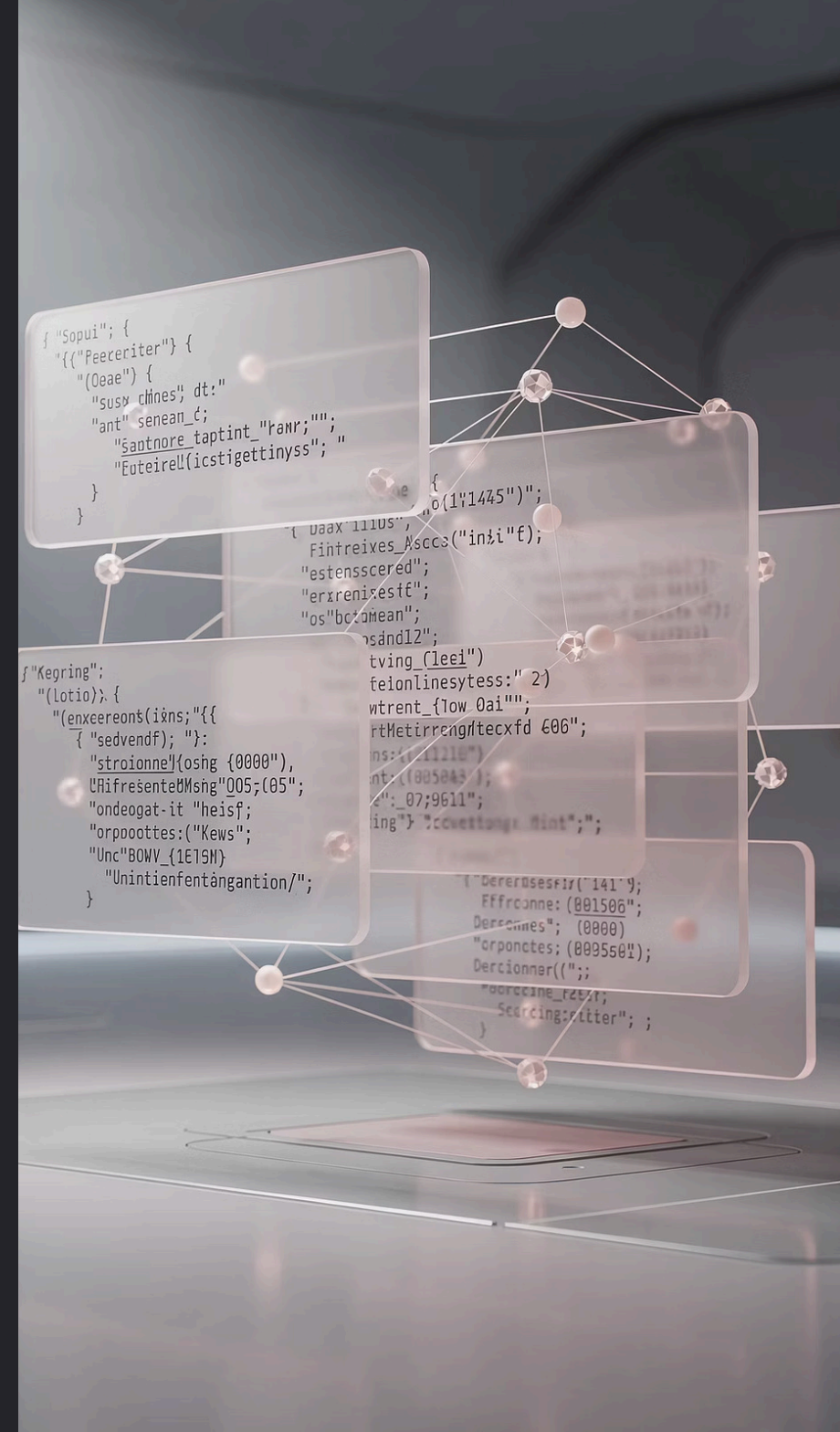
Colección: `products_catalog`

Implementa el **Attribute Pattern**: los atributos variables por categoría (p. ej., talla para ropa, voltaje para electrónica) se almacenan como subdocumentos, evitando columnas nulas.

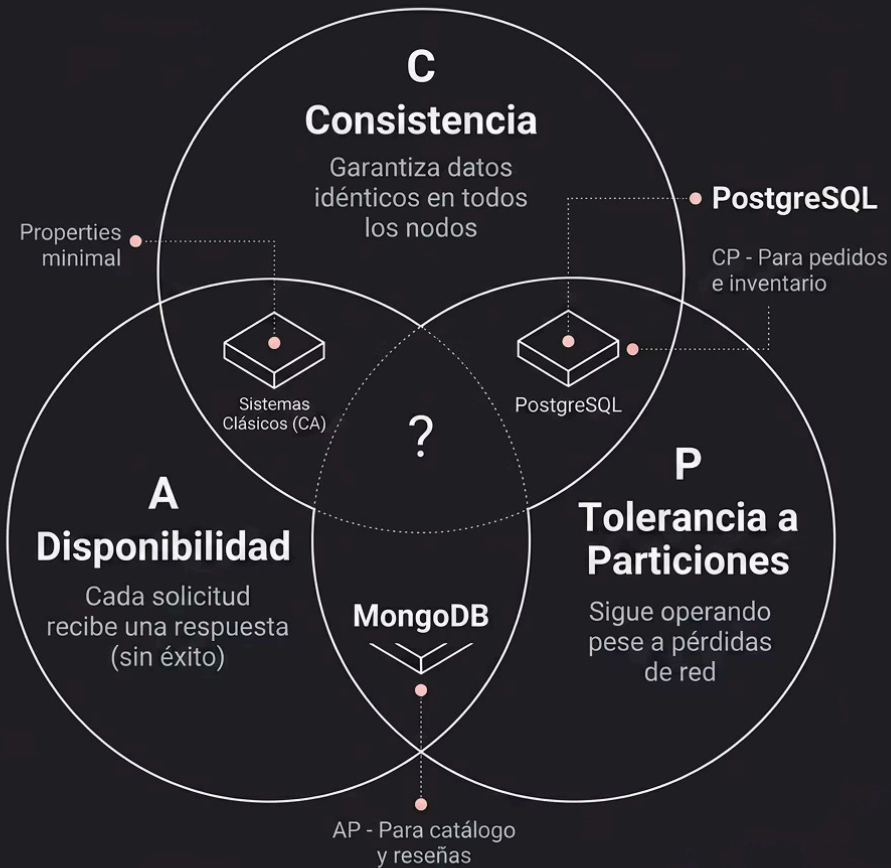
Colección: `order_reviews`

Implementa el **Extended Reference Pattern**: incluye los campos más consultados del pedido directamente en el documento de reseña, eliminando joins costosos en consultas de analítica masiva.

- ✅ Ambos patrones reducen la latencia de lectura al estructurar los datos según los patrones de acceso reales del negocio.



Teorema CAP y Sincronización CDC



Posicionamiento CAP

- **PostgreSQL** → **CP**: Prioriza consistencia para proteger cobros e inventario crítico
- **MongoDB** → **AP**: Prioriza disponibilidad para mantener el catálogo siempre en línea

Sincronización en Tiempo Real

La arquitectura CDC lee el **WAL de PostgreSQL** mediante **Debezium**, publica eventos en **Apache Kafka** y los consume MongoDB para mantener coherencia eventual entre sistemas.

Capturar WAL

Publicar en Kafka

Consumir en MongoDB

Limitaciones, Mitigaciones y Conclusión

Trade-off Principal

Administrar dos motores de base de datos más un bus de eventos incrementa significativamente la **complejidad operativa**: despliegue, monitoreo, backups y troubleshooting deben coordinarse entre sistemas distintos.

Mitigación Estratégica

- **AWS RDS** para PostgreSQL gestionado
- **MongoDB Atlas** para el motor documental
- **Confluent Cloud** para Kafka serverless

Conclusión Ejecutiva

Ecommify demuestra que una arquitectura políglota bien diseñada permite construir un sistema de comercio electrónico:

- **✓ Escalable** a millones de transacciones
- **✓ Seguro** con integridad ACID donde importa
- **✓ Altamente disponible** en catálogo y analítica
- **✓ Mantenable** con servicios cloud gestionados

